

Fine-grained Communication Phase based Analytical Performance Modeling and Analysis

Vishal Deka

Department of Computer Science and Engineering
Indian Institute of Technology Kanpur
Kanpur, India
vdeka@cse.iitk.ac.in

Preeti Malakar

Department of Computer Science and Engineering
Indian Institute of Technology Kanpur
Kanpur, India
pmalakar@cse.iitk.ac.in

Abstract—High-performance computing is essential for scientific innovation. With the advent of exascale computing and the growing scale of scientific workloads, novel tools and methodologies are extremely important to analyze and model the performance of large-scale scientific applications. Existing profiling and tracing tools have certain limitations: profiles do not allow fine-grained performance modeling, whereas trace-based simulation or performance modeling is expensive and often infeasible for large applications. In this work, we propose fine-grained communication phases as a level of abstraction for analyzing and modeling application performance. Given the iterative nature of HPC applications, we propose a methodology to automatically group MPI communication events into phases and identify the unique and repeating phases. Our approach enables modeling only the unique phases, in contrast to a complete trace simulation. Further, we address the limitation of existing analytical communication models to model run-time delays in communication time. We propose a new delay-aware communication model (DACM) that achieves a best-case prediction error of 10.11% across six diverse HPC benchmarks and applications, in contrast to 53% using existing analytical models.

Index Terms—high performance computing, performance analysis, performance modeling

I. INTRODUCTION

High performance computing (HPC) enables execution of compute-intensive large-scale scientific simulations. Message Passing Interface (MPI) has been the de facto programming model for parallel applications. As applications grow in scale and complexity, there is an increasing need for novel tools and methods to analyze and accurately model the performance of parallel applications. Performance analysis may help application developers debug performance bugs on newer systems. Performance modeling [35], [41] helps in estimation of application runtime without executing the application. Traditionally, developers use profilers to collect application performance data and model performance using trace-based and discrete-event simulations, analytical modeling or statistical modeling [19], [28], [41], [46].

Profiling tools (TAU [40], HPCToolkit [6]) provide aggregate performance statistics such as the maximum time spent in an MPI function. However, they do not provide sufficient detail for performance modeling. Tracing tools [33] provide timestamped logs of each MPI function invoked by the application

including details such as message size and source-destination pairs. This enables tools such as LogGOPSIm [28] to model or simulate each MPI event in the trace using analytical models [21], [25], [27]. However, traces of HPC applications can be huge, of the order of gigabytes [9], [29], [49]. This makes it difficult to analyze them using traditional tools. It also makes trace-based performance modeling time-consuming. Further, the analytical models used for trace-based performance modeling do not consider the impact of late arrivals of ranks due to execution-time delays that in turn impact the application communication times. Such delays may originate from system-level factors such as network congestion, message ordering by application and MPI implementation, scheduling policies, node allocation which are challenging to model using deterministic analytical models.

Our work addresses the lack of finer detail in profiles and the challenge of generating, analyzing and modeling using large traces. We propose performance analysis and modeling of parallel applications at the granularity of communication phases. We develop an intermediate fine-grained profiling technique that contain sufficient detail in order to model the application communication times. Typical scientific programs iteratively perform the same computation and communication routines across several time steps as desired by the user and as required by the application to solve the overall scientific problem. For example, in a simple parallel stencil code, a stencil computation routine is followed by a halo-exchange routine in every time step. The sequence of MPI communication events in the halo-exchange routine forms a communication phase. In this paper, we propose a heuristic to automatically identify all the communication phases in an application. We then use a clustering algorithm to find the unique and repeating phases among them. This enables us to model only the unique phases, as opposed to a complete trace simulation, as in the case of tools such as [24], [28], [46].

Our next contribution is to model the run-time delays in MPI communications that impact the communication time of HPC applications. We propose a delay-aware communication model (DACM) to model the delays in communication times. We use the data collected from a single run of the application to model the delays or late arrivals for other input configurations and process counts. The delay values are scaled according to the

problem size and number of ranks. We use these delay values to predict the point-to-point and collective times of different input configurations. The phase identification process provides a set of unique phases and their frequency of occurrence in the application. Combined with the analytical models and our proposed DACM, we compose a complete communication model of the application. We evaluated our work on a set of six HPC benchmarks and applications - NAS-LU, HPCG, LAMMPS, miniAMR, Quicksilver and Xcompact. Across all the applications, we achieved a minimum model error of 10%. We achieved average prediction error of 68.65%, 14.62%, 19.37%, 24.11%, 10.11% and 69.49% on NAS-LU, HPCG, LAMMPS, miniAMR, Quicksilver and Xcompact were respectively. We achieved $\sim 3\times$ lower errors than the baseline analytical models.

The contributions of this work are

- We propose a method to automatically identify communication phases from MPI traces.
- We augment analytical communication models to address run-time delays and propose a delay-aware communication model.
- We combine communication phase identification with communication models to develop a novel way of modeling application performance.
- Finally, we evaluate and analyse the proposed methods on a set of six diverse HPC applications.

The overall structure of this paper is as follows: Section 2 gives a summary of the relevant literature, Section III describes the phase identification methodology, Section IV explains the various performance models including our proposed model and Section V evaluates and analyzes our methodologies on a set of HPC applications.

II. RELATED WORK

In this section, we present a review of the literature relevant to our work. Several popular tools have been developed for profiling and tracing high-performance computing (HPC) applications, ranging from light-weight libraries to complex, end-to-end software packages. Simple profiling and tracing tools such as mpiP [47], DUMPI [38], provide raw performance data, while large packages such as TAU [40], HPCToolkit [6], Vampir [33], Scalasca [48] provide additional features, including visualization of traces, call-site profiling, root-cause analysis, among others. Our tool differs from the existing ones in the way we process the raw traces. While other tools help visualize or analyze the raw traces, we propose a method to find repeating communication phases from the traces and build performance models based on the identified phases.

The notion of *communication phase* has been used in varying ways in the literature. Alawneh et al. [10] defined phases as the initialization, solve, and finalization phases of scientific programs and used information theory to identify phases. Jin et al. [32] consider time steps in HPC applications as phases and proposed a string-matching technique to identify them. The authors of [5] proposed a source-code-centric view of phases wherein each phase is uniquely identified by the function being

executed during that phase. In contrast, we consider phases at a finer granularity that comprises communication phases. This helps in building analytical models using a single reference run. Similar to ours, Isaacs et al. [31] identify fine-grained communication phases using a graph algorithm, however they use phases for performance visualization.

We now survey existing analytical performance models. Analytical communication models such as the Hockney model [27], LogP model [21] have formed the foundation of performance modeling of HPC applications. Over the years, improvements to these models have been proposed [25], [11] to adapt to changing architectures and network topologies. These point-to-point models were further used to construct performance models for collective communications in [36], [43], [44]. These models are used in trace-based simulation and performance modeling of HPC applications by tools such as DIMEMAS [24], MFACT [46], LogGOPSim [28]. Choi et al. [18] used the Max-rate model [25] for trace-based modeling of MPI-based GPU applications. Similarly, LogGOPSim [28] and [39] make use of the LogGP [11] model, an improvement over the LogP model for trace-based performance modeling. Our work is different from trace-based modeling – we do not require a full-scale simulation or modeling of the application. Instead, we only model the unique and repeatedly occurring communication phases and compose a communication model for the application using these models.

Another branch of work has focused on analytical modeling of HPC applications based on knowledge of the source code [13], [22], [42]. While this approach does not rely on traces, it requires deep domain knowledge to build such models. Yet another branch involves the use of data to model application performance using machine learning models [7], [14], [16], [30], [34]. Here, the application is run several times with different input configurations and process counts to build a dataset mapping application input parameters to execution time, and machine learning models are used to predict the execution time. However, unlike machine learning models, we do not need large datasets. Our proposed delay-aware communication model requires data from only a single previous run of the application.

III. COMMUNICATION PHASE DISCOVERY

Profiling and tracing tools have been widely used for analyzing and modeling the performance of HPC applications. While profiles provide summarized information, traces provide detailed logs of communication events, thus enabling trace-based simulation or performance modeling [19], [28]. However, as applications grow in scale and complexity, large trace files are difficult to analyze and model [9], [29], [31], [49]. In this paper, we propose identifying and using communication phases for performance analysis and modeling.

Figure 1 shows an overview of our workflow, consisting of (1) identification of communication phases, (2) mapping communication phases to unique clusters, and (3) performance modeling of the identified clusters. The phase identification process begins with a timestamp-based heuristic to group MPI

events into communication phases (Section III-B). Next, we use a clustering algorithm on the sequence of identified phases to map them to a set of distinct and repeated clusters (Section III-C). A cluster, in this context, refers to unique phases that repeat across timesteps. Next, we model the communication time of clusters using analytical models and finally build a complete model of the application's communication time (Section IV). Our performance models are focused on communication time, as the computation time can be modeled using simple linear models [7].

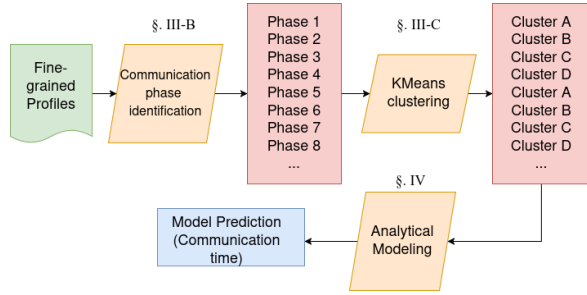


Fig. 1: Overview of framework workflow – communication phase identification, clustering and modeling.

A. Fine-grained Communication Profiles

High performance scientific applications are designed to solve scientific problems iteratively (e.g. using numerical integration). Thus, a typical parallel application comprises computation and communication phases, and the durations of these phases vary with the execution configuration. Several parallel application profiling and tracing tools such as TAU, Intel VTune, HPCToolkit exist [2], [3], [6], [33], [40] that generate communication profiles (summary) or detailed timestamped traces. We introduce fine-grained profiling (communication phases) as an intermediate level of abstraction for analyzing and modeling HPC application performance.

We define a communication phase as a short sequence of MPI communication calls invoked in close temporal proximity, i.e., they are likely to overlap in time (in case of non-blocking calls) or occur almost simultaneously. These communication phases may be related to communication routines, e.g., halo exchange function comprising MPI_Send and MPI_Recv calls. Thus, our definition of a communication phase (henceforth referred to as a phase) is intended to closely represent the communication-intensive routines in the source code, similar to [31]. In this paper, our goal is to automatically identify the communication phases from the MPI traces. We use the IPMPI tool proposed in [8] to generate the traces, which record all individual communication operations along with message sizes and timestamps, including the point-to-point communications of MPI collectives. The communication phases are visible as bursts of MPI activity as demonstrated in Figure 2. The figure shows four unique communication phases (A, B, C, D) of varying time duration and two timesteps (i and $i+1$), separated by the blue dotted line. The figure is representative of a

10-second communication trace. The reddish points represent individual MPI events. Y-axis shows the time taken by these events. It can be observed that the MPI events occur in bursts. These short bursts of communication represent our notion of communication phase, i.e., a sequence of communications in close temporal proximity. The black rectangles in the figure are used to highlight the different phases. The four distinct phases are observed to repeat in both timesteps. This is representative of the fact that the timesteps in an HPC application are generally identical in terms of the source-code semantics as well as execution times (subject to typical variability in time). In the figure, the empty spaces in between any two phases may represent computation where no MPI activity is seen.

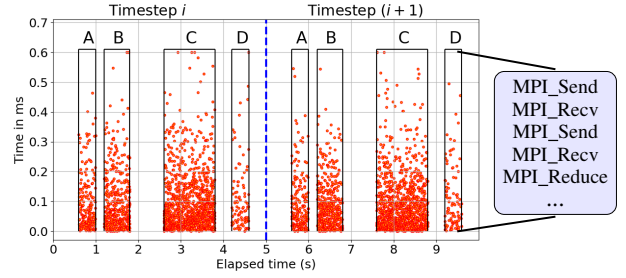


Fig. 2: Two timesteps of a representative trace showing MPI communication events (red) and phases outlined using black rectangles.

Figure 2 shows 8 communication phases (A, B, C, D, A, B, C, D). Out of these, there are 4 distinct phases (A, B, C, D) that repeat across timesteps. To avoid ambiguity, we use the term *cluster* to refer to distinct phases. For example the first phase of timestep i and the first phase of timestep $i+1$ are mapped to cluster A. In this figure, all 8 phases are uniquely mapped to one of the *clusters* A, B, C and D. A real application may have any number of clusters (short communication phases) per timestep. Formally, a sequence of M phases within a communication trace is defined as $\pi = [p_1, p_2, \dots, p_M]$, equivalent to the 8 phases in Figure 2. Further, let $P = \{\phi_1, \phi_2, \dots, \phi_N\}$ be the set of N clusters or unique communication phases. Each phase p_i in π can be uniquely mapped to a cluster ϕ_j in the set P . In other words, a cluster ϕ_j is expected to repeat in every timestep. In Section III-B, we explain our heuristic to group individual MPI communication events from the trace into the sequence of phases π . Note that our method assumes no prior information of the source code or the start and end times of phases. We explain the mapping of phases to clusters in Section III-C.

B. Automatic Identification of Phases

Our first step towards performance modeling is to identify communication phases. One of the main challenges involved with automatic phase identification is that we do not have any prior knowledge of application source code. Therefore, we do not have information about the start and end times of the communication phases or timesteps. We wanted to develop an application-agnostic approach, therefore we did not modify

the source code to obtain this information. Thus, our approach is designed to automatically identify the timestamps associated with the beginning and ending of phases. The main motivation of our phase identification heuristic is that if two MPI events in the trace are separated by a sufficiently large time duration (we denote this as threshold), then they can be considered to belong to two different communication phases. A sufficient gap in time between the occurrence of two events may indicate a computation phase during that time. This is represented by the white spaces between consecutive phases in Figure 2. The primary challenge is to determine this threshold. The time taken by computation between two communication phases is application-dependent. We found this to be in the range of tens of milliseconds. In Figure 2, the gap between clusters A and B is much smaller compared to that between B and C. A large threshold would club A and B together, which is not desirable. Therefore, an appropriate threshold is required that correctly separates out the events in A and B into two different phases. However, finding this threshold value in a real trace is not straightforward because we do not assume any prior information about the source code of the application. Next we explain our methodology.

First we compute the difference in timestamps between every pair of consecutive MPI events $(i, i+1) \forall i$ in the trace. Let us denote this difference by δ_i . If δ_i is above a threshold, Δ , we group the events at i and $i+1$ into two different phases, otherwise we include them in the same phase. To determine the threshold Δ , we employ the following heuristic. We bin the δ_i values for all successive events $(i, i+1) \forall i$. The first bin (i.e. with lowest delta values) will most likely contain all the δ_i values associated with events in the same phase. This is because the communication events within a phase occur in close temporal proximity. Therefore, the δ_i values of pairs of successive events in the same phase are smaller than those between two events belonging to two different successive phases. This is depicted in Figure 2, where the red points within a phase are very close to each other, but the boundary points in two different successive phases are farther apart. The second bin onward contain the δ_i values that belong to events separated by a larger time gap. Therefore we select the timestamp value of the second bin as our threshold Δ to separate two successive events into different phases. Thus we obtain the sequence of phases π . Next, we describe our methodology to find the set of unique clusters, P .

C. Discovering Unique Phases

The methodology described above provides us with a sequence of communication phases as shown in Figure 2 (labeled as Phase 1, Phase 2, ...). However, although we know that these phases repeat during every timestep, we are unaware of the exact repetitive pattern of communication phases because we do not assume application source code knowledge in this work. Thus the next step in our framework is to identify the set of repetitive phases which we term as cluster as shown in Figure 2 (labeled as Cluster A, Cluster B, ...). Now we explain our methodology to map each phase p_i to a cluster from

$P = \{\phi_1, \phi_2, \dots, \phi_N\}$ along with the associated challenges. Note that our phases are identified based on the timestamps. Thus usual variation in system performance throughout the application execution may cause differences in timestamps, i.e. some events may take longer time to complete. This may introduce certain inconsistencies, such as, certain instances (phases) of a particular cluster may be wrongly split into two consecutive phases due to high difference in the timestamps. Thus, we employ a clustering method to map the sequence of phases to unique clusters.

The rationale of clustering is that phases with similar communication behaviour belong to the same cluster. For example, a single timestep of a program may have a point-to-point based halo-exchange phase and a reduce phase. Thus, all the halo-exchange phases in the program must belong to a single cluster, while all the reduce phases must belong to a different cluster. To achieve this, we extract key characteristic features from each phase. The features include: total number of MPI calls, the number of point-to-point (P2P) and collective operations, the average data size exchanged during that phase, the amount of computation time before this phase started and the mean and maximum duration of events during the phase. These features are expected to reflect the communication behaviour of the phase. We obtain the feature vectors corresponding to the phases π . Now, we apply the K-Means clustering algorithm on these feature vectors to obtain the set of clusters P (repeating phases). We also record the number of times each cluster appeared in the sequence to obtain the count of repetitions of each phase $\phi(n)$, as described in Section III. Thus, at the end of our phase identification process, we obtain a set of distinct phases P . In the next section, we discuss our methodology to build analytical performance models based on the characteristics of the identified phases.

IV. ANALYTICAL MODELS FOR PHASES

This section details the performance modeling of the clusters identified in the previous sections.

In this work, we propose a novel way of modeling applications by modeling only the unique and repeating communication phases or clusters, as described in Section III. This is in contrast to full-scale simulation techniques [19], [24], [28], [37]. In Section III-C, we mapped the sequence of phases π to the set of clusters P . Assume that each instance of $\phi_i \in P$ is repeated $\phi_i(n)$ times in the sequence π . Let the time taken by a single instance of the cluster ϕ_i be $T(\phi_i)$. Therefore the total time taken by the cluster ϕ_i is $T(\phi_i) \times \phi_i(n)$. Thus, the total communication time of the application is the sum of the time taken by all the instances of all the clusters, i.e., $\mathcal{T} = \sum_{\phi_i \in P} T(\phi_i) \times \phi_i(n)$. $T(\phi_i)$ can be expressed as the sum of times taken by P2P and collective calls occurring in that cluster (unique communication phase). Next, we explain modeling point-to-point (P2P) and collective communications.

A. Modeling Point-to-point Communication

In this section, we explain the performance models for point-to-point communication. We first explain our extended

Hockney model, which adapts the latency and bandwidth parameters to different network types. We further refine this model to account for latency and bandwidth variations caused by network congestion.

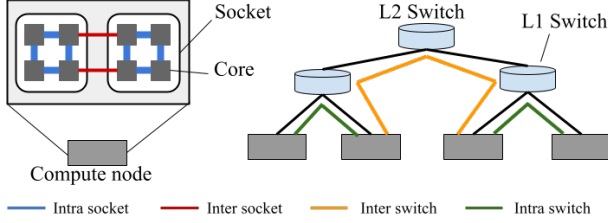


Fig. 3: Node topology with 2 sockets, 4 cores per socket (Left). Mini cluster with 4 nodes connected in 2-level fat-tree network topology (Right). Different communication channels are shown in different colors.

1) *Extended Hockney Model (Basic)*: Modern compute nodes in HPC clusters have multiple sockets, with each socket housing multiple cores, and these nodes are interconnected by high-speed networks. Figure 3 shows a schematic of a typical compute node with two sockets and multiple cores per socket and a mini cluster of four compute nodes. MPI ranks communicating within a socket perceive different latency and bandwidth as compared to ranks communicating across sockets. Similarly, ranks placed on different nodes communicate over the network, possibly across multiple switches, incurring higher latency and lower bandwidth. We account for this heterogeneity and distinguish among communicating pairs based on the channel (network type) of communication (intra-socket, inter-socket, intra-switch (within with the same leaf switch as shown in green), or inter-switch (shown in orange)).

We develop our analytical communication models on the two-parameter Hockney model [27]. The model estimates the time taken by a message of size M as $T(M) = \alpha + \beta M$ where α and β are network latency and reciprocal of bandwidth respectively. However, latency and bandwidth are dependent on message size [7] and the communication channel/type of network (shown in Figure 3). We extend this equation to include both factors. Thus, for every communication of message size M , transmitted over channel c (intra-socket, inter-socket or inter-node), we estimate the communication time using an extended Hockney model.

$$T(M, c) = \alpha_M^c + \beta_M^c M \quad (1)$$

We refer to the model in Equation 1 as the **Basic** model. We empirically determined α and β for each network type and message size using the Intel MPI benchmarks¹ on the target platform. Thus we maintain a lookup table for 24 message sizes and the corresponding latency and bandwidth values, per network type. We calculate the α and β values using linear interpolation for the message sizes whose parameters were not empirically determined.

¹<https://github.com/intel/mpi-benchmarks>

2) *Congestion Model*: The basic model does not consider the network congestion occurring due to simultaneously communicating ranks. Congestion may occur in the intra-node and inter-node network links due to shared resources (caches, memory channels, network interface). The available bandwidth is shared due to congestion i.e. when multiple ranks send/receive data over a network (intra-node or inter-node). Thus, we refined the basic model to capture network congestion. We ran the Intel MPI micro-benchmarks with a varying number of ranks per node to emulate congestion, separately for each network type. This results in a new set of latency and bandwidth values $\alpha_M^{c,k}$ and $\beta_M^{c,k}$, for message size M , channel c , and a new parameter k which determines the number of ranks competing for the channel c . We maintain latency and bandwidth values for each process count per node, ranging from 1 to the available number of cores. Thus for communication of message size M bytes, competing with $k-1$ other ranks for the channel c , the time taken is expressed as

$$T(M, c, k) = \alpha_M^{c,k} + \beta_M^{c,k} M \quad (2)$$

We refer to this model as the **Congestion** model. We also compare our models with the state-of-the-art Max-rate model [25], which was an improvement over the Hockney model.

We assume that the P2P time of a cluster is determined by the slowest rank. We model every MPI_Send performed by every rank using Equation 2 and sum up the estimated times. The time taken by the cluster is thus determined by the maximum time across all ranks. We also need to determine parameter k (number of competing ranks on a network channel) used in Equation 2 and Max-rate model. It is not straightforward to accurately determine the exact degree of overlap among pairs of processes. This is because the messages from different processes may arrive randomly at the network interface, shared caches, or on the cross-socket link depending on system parameters and the application. Prior work [15], [22] assumed a fixed amount of congestion for each channel depending on the hardware. We refine this idea by also considering the application's communication pattern. We construct a communication matrix for each cluster that provides an estimate of the number of ranks using a particular network type/channel. Accordingly, we use this count to determine the degree of congestion/overlap (k) for the communication models.

B. Modeling Collective Communication

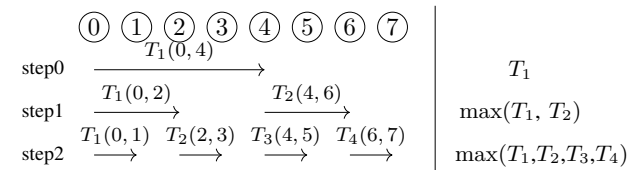


Fig. 4: Steps for binomial algorithm of MPI_Bcast. $T_i(s, r)$ is the time taken by P2P message between ranks s and r .

MPI collectives are composed of multiple point-to-point (P2P) communications. The communication order and the

number of communication steps are determined by the MPI collective algorithms implemented in the MPI library. Each communication step may comprise one or more concurrent P2P among all or a subset of ranks. Data size of communication in every step may be same or different depending on the collective. We obtain detailed P2P communications (ranks and message sizes) per communication step of a collective using IPMPI [8], which also considers the collective algorithm used. We exemplify the binomial algorithm for broadcast [44] in Figure 4. We estimate the time taken by a communication step as the maximum of all P2P communication times in this step, assuming they occur concurrently. A step may not start before the previous step completes due to dependencies. We estimate the time taken by the collective as the sum of the times taken by all the steps. We also pictorially represent this on the right side of the figure. In general, let n be the number of communication steps in a collective and S_t be the set of P2P communications in the t^{th} step. We estimate the individual P2P time $T_{P2P}(i)$ of all P2P communications in S_t using the models described in Section IV-A based on the sender, receiver and message size. We estimate collective communication time using

$$CT = \sum_{t=0}^n \max T_{P2P}(i) \quad \forall (i) \in S_t \quad (3)$$

C. Delay-aware Communication Model (DACM)

The deterministic analytical models described in Sections IV-A and IV-B do not take into account the run-time delays in message transmission. In case of collective communication, these models assume that all ranks initiate P2P communications simultaneously. Similarly, for point-to-point, the models assume that MPI_Send and the corresponding MPI_Recv are posted at the same time. However, this may not be the case in practice. We empirically observed that a small percentage of the communications incur large delays (Figure 5), thus rendering the simple analytical models ineffective because they model only ideal communication times. Deterministic models may not be able to account for the wait times incurred due to variability and late arrivals. Thus, we include the variation in wait times in our model. We describe this next.

We propose a Delay-aware Communication Model or **DACM**, for collective and P2P communication that takes into account the run-time delays. The motivation for developing this model stems from the observation that for all process counts and input configurations of an application, the delays follow a certain distribution as shown in Figure 5. The figure shows the measured times (Y-axis) of all collective communications in sorted order (X-axis) for four different configurations of LAMMPS and miniAMR. It is clearly visible that there are three categories of communication times (demarcated by the vertical lines). The similarity in categories is evident across all configurations in both applications. We also observe that there are certain communications that take fairly longer times (right of the rightmost black dotted vertical line) and certain communications take less times (left of the leftmost black dotted vertical line). This also suggests that it may be

possible to model the delayed arrivals leading to delay in communications. We elaborate on our delay estimation method next.

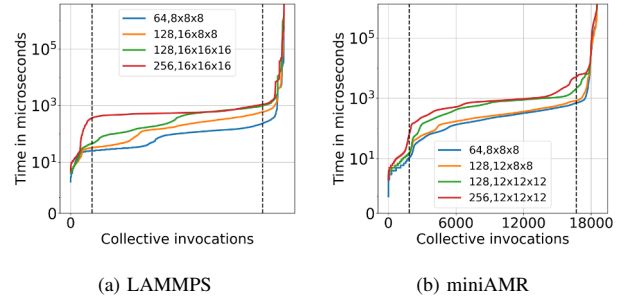


Fig. 5: Delays incurred in MPI collectives, sorted by duration, for various configurations across two applications.

First, we calculate the delays for every point-to-point (P2P) and collective function from an execution of the application. This execution configuration is chosen from a smaller process count/problem size, we denote this as the *reference run*. The collective function time is determined by the slowest rank which may delay other ranks because all ranks participate in a collective. Thus we estimate the delay of a collective function by measuring the difference in time taken by the slowest rank and the fastest rank from our reference run. Note, we do not use the individual P2P to estimate the variation in communication times (delay) for collectives. It is challenging to determine delays in point-to-point communications due to possible chains of dependency as studied in [48]. We estimate the delay as the difference between the measured time and the ideal communication time estimated by the basic Hockney model (Equation 1).

We describe next our methodology to estimate delays incurred by P2P and collectives of a new configuration of the application. First, we estimate the communication time of every communication using the basic model described in Sections IV-A and IV-B. Next, we need to estimate the delays in the communication due to some late arrivals using the delays measured from the reference run. The challenges in this estimation are that the new configuration may have (1) a different number of function invocations and (2) the arrival time delay of the ranks may be different than the reference run. We require an estimation of the count of all communications (P2P and collectives). We can either use a historical trace of this configuration collected from a different machine (may even be a single node) or we can execute for only 1 timestep if the trace is not available (on a single node). Note that we need to run only 1 timestep because we correctly identify the phases and the iterative nature of HPC applications. We further assume that the arrival time delays in this new configuration scale differently than the reference run. However, note that this arrival time delay depends on the process count and the problem size because the problem size determines the compute phase time. The compute time directly determines the arrival

time of ranks at the communication phases. Therefore, for a new configuration with a different problem size, we estimate as follows. We scale using the arrival time delay incurred at the reference run and the ratio of the new problem size to the reference problem size. Similarly, for a new configuration with a different process count, we scale using a factor 1.2 of the arrival time delay incurred at the reference run and the ratio of the new process count to the reference process count. This is because we empirically observed that process count has a larger impact on the arrival time delays than the problem size. We estimate the DACM model time T_i of i^{th} P2P call by adding the Basic Hockney model time (Equation 1), i.e. $T(M, c)$, and the delay d_i for the i^{th} message. We add the subscript i to equation 1 to identify the i^{th} communication.

$$T_i = T_i(M, c) + d_i \quad (4)$$

Similarly, the delay d_i is added to the Basic model time (Equation 3), i.e. CT_i , to derive the DACM model time of the i^{th} collective call.

$$CT_i = CT_i + d_i \quad (5)$$

V. EXPERIMENTAL EVALUATIONS

In this section, we first describe the platform and applications, followed by effectiveness of our phase identification method and evaluation of performance models. We also compare our models with machine learning regression models and show the generality of our models using a cross-platform study.

A. Experimental Setup and Applications

We conducted our experiments on the Intel Xeon Cascade Lake based supercomputer in our institute. The 48-core nodes are connected using Mellanox (HDR) InfiniBand interconnect network (100 Gbps) in a fat-tree topology. We compiled the applications using Intel MPI compiler and experimented on 2 to 16 nodes of the cluster, with 32 MPI ranks per node. We generated the application fine-grained profiles using IPMPI [8], which is based on the PMPI profiling interface. We implemented phase identification as well as performance models in Python.

Application	Inputs parameters	P2P Time%	Coll Time%
NAS-LU	Class C,D,E	0.3	0.01
HPCG	Matrix size nx,ny,nz	0.69	2.28
LAMMPS	Number of atoms	9.25	0.92
Quicksilver	Number of particles	NIL	8.49
miniAMR	Mesh size nx,ny,nz	10.15	34.88
Xcompact	Mesh size nz,ny,nz	NIL	45

TABLE I: Characteristics of different applications.

We experimented on six diverse applications/benchmarks – LU, HPCG, LAMMPS, Quicksilver, miniAMR and Xcompact. Table I shows the input parameters and percentage of P2P and collectives in each application. The LU benchmark from the NAS Parallel Benchmarks suite [12] is a simplified fluid dynamics solver and mainly uses small-sized blocking P2P

calls. HPCG [26] is a benchmark used to rank supercomputer performance and uses blocking P2P and MPI_Allreduce calls. LAMMPS [45] is a popular molecular dynamics application. It extensively uses P2P calls and MPI_Allreduce. Quicksilver [1] is a Monte Carlo transport benchmark, part of the CORAL-2 benchmark suite. It uses non-blocking P2Ps and MPI_Allreduce for communication. miniAMR [20] is a mini-application from the Mantevo benchmark suite. It can be configured to use blocking/non-blocking P2P communication and uses MPI_Allreduce. Finally, Xcompact [4] is a finite-difference flow solver for computational fluid dynamics tasks. It exclusively uses MPI_Alltoall calls, thus providing a good test case for studying collective communication performance.

B. Phase Identification Results

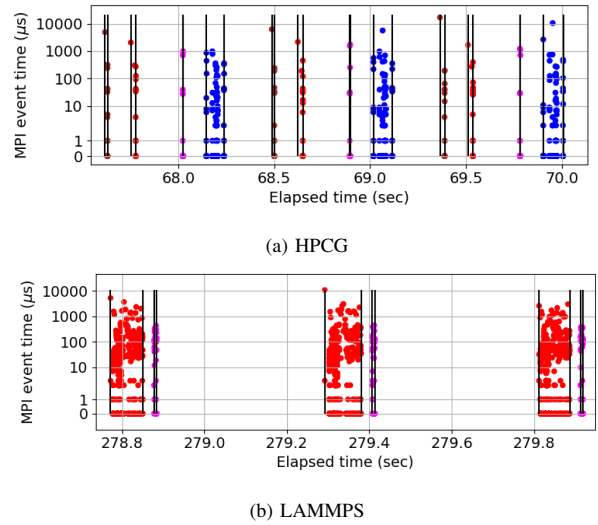


Fig. 6: Short snapshots of HPCG and LAMMPS showing identified clusters in different colors.

Figures 6a and 6b show short snapshots of traces and identified clusters for HPCG and LAMMPS. X-axis represents the elapsed time, scatter plots show the time taken by individual MPI communication events during this time and the black vertical lines show the boundaries of identified phases using our approach similar to Figure 2. Distinct clusters are shown in distinct colors, same clusters in different timesteps are shown in the same colour. We observe that our method clearly identifies the communication clusters in all time steps for both applications. We observed this for the other applications as well. This is an important contribution of our work because we estimate the communication times using the performance models based on these identified repeated communication clusters. We observe four clusters in the case of HPCG for each of the three timesteps shown in Figure 6a. The cluster with the longest timespan is shown in blue, and there are three other shorter clusters in each phase. We analyzed the blue cluster and found that it consists of point-to-point calls (MPI_Send, MPI_Irecv and MPI_Wait) while the other

Application	#MPI Calls/timestep	Basic		Max-rate		Congestion		DACM	
		MAPE	MedAPE	MAPE	MedAPE	MAPE	MedAPE	MAPE	MedAPE
NAS-LU	48K-1.2M	67.47	43.86	53.77	42.86	53.77	42.86	68.65	51.0
HPCG	40K-140K	96.82	96.9	94.92	95.02	94.92	95.02	14.62	13.46
LAMMPS	10K-50K	90.08	90.88	76.01	75.66	76.01	75.66	19.37	18.01
miniAMR	60K-0.4M	97.97	98.05	96.64	96.84	96.64	96.84	24.11	18.28
Quicksilver	2K-16K	99.94	99.95	99.94	99.95	99.94	99.95	10.11	9.08
Xcompact	3K-10K	50.55	39.93	-	-	-	-	69.49	59.47

TABLE II: Mean absolute percentage error (MAPE) and Median absolute percentage error (MedAPE) for various models, across different applications

clusters contain both MPI_Allreduce and a few point-to-point calls. We observe two clusters in LAMMPS (Figure 6b). The red cluster consists of point-to-point calls, while the magenta cluster consists of P2P and MPI_Allreduce calls.

C. Analytical Model Performance

In this section, we evaluate our analytical performance models. We compare and contrast four models – basic model (Basic) in Equation 1, the state-of-the-art Max-rate model [25], Congestion model in Equation 2, and Delay-aware communication model (DACM) explained in Section IV-C. The Max-rate model is expressed as $T(m, k) = \alpha + kM/\min(R_N, kR_C)$ where k is the number of simultaneously communicating processes, R_N is the maximum available bandwidth of the channel, and R_C is the bandwidth sustained by a single process. We adapt this model for the different network types similar to our basic model (Section IV-A) and as done in [18] for fair comparison because by default Max-rate considers uniform bandwidth values.

We show the mean absolute percentage error (MAPE) and Median absolute percentage error (MedAPE) of Basic, Max-rate, Congestion and DACM models in Table II (columns 3 – 6) for six applications on 64 to 320 processes across 2 to 10 nodes respectively. The error is computed using the modeled and measured communication time of the entire application execution across varying input configurations (problem sizes and process counts). The execution times of the applications varied from 1 – 45 minutes, depending on the inputs and process counts. Lowest execution times were observed for NAS-LU benchmark. We also show the range of the number of MPI function calls per timestep (column 2) across all input configurations per application. LAMMPS, LU, HPCG and miniAMR use blocking P2Ps along with collectives, while Quicksilver and Xcompact only use collective calls. The table shows the lowest error obtained for each application across models in green colour. We observe that DACM results in 14.6-96.8%, 19.3-90.1%, 24.1-97.9, 10.1-99.9% lower errors for HPCG, LAMMPS, miniAMR and Quicksilver. This is because DACM handles the variation in start times of different ranks of the applications as shown in Figure 5 along with network-based modeling. The reference run for DACM was on 64 processes for all applications. We observed a high error margin (more than 100%) for Xcompact using Max-rate and Congestion models, hence these values are not shown in the table. DACM also exhibits high errors for Xcompact. The reason is that MPI internally uses non-blocking Isend/Irecv

calls for MPI_Alltoall with large message sizes as is the case with Xcompact. However, we modeled collectives using blocking P2Ps (Section IV-B). We show the detailed results of each application next.

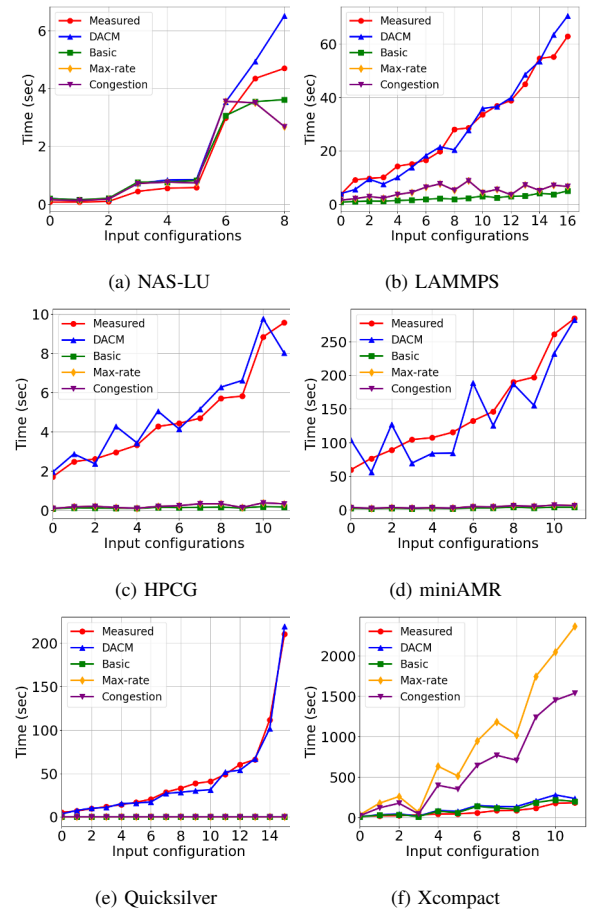


Fig. 7: Measured and modeled total communication times for different applications across input parameter configurations.

Figure 7 shows the comparison of measured and modeled times for various input configurations (x-axis) for all six applications. We observed that for most configurations of LU, the predictions are quite close to the measured, barring a few configurations where DACM over-predicted and others under-predicted the times. However, note that the absolute error is about 1 second. We observe that DACM modeled times

(blue) closely follow the measured times (red), whereas the Basic, Max-rate and Congestion models exhibit high errors for almost all configurations of all applications because these models are deterministic and do not consider run-time delays. DACM achieves lowest MedAPE of 9.08% for Quicksilver and highest MedAPE of 59.47% for Xcompact. We observe a slight improvement with Max-rate and Congestion model for LAMMPS. This is because the network bandwidth saturates due to multiple large P2P communications in LAMMPS, thus the Max-rate and Congestion models are effective because they consider the effective drop in bandwidth due to simultaneous communications. The applications HPCG, LAMMPS, miniAMR and Quicksilver use Allreduce function calls with small message sizes. There is a huge difference in MPI communication times incurred by each rank due to late arrivals of some MPI ranks. The delayed arrivals may arise from various system and application-related factors. DACM models the delayed arrivals for both P2P and collectives, therefore we observe better DACM model estimates as shown in Figures 7c, 7d, 7b and 7e.

D. miniAMR Case Study

We now elaborate the impact of delays in communication time using miniAMR. We chose this application because it is communication-intensive and uses both P2P and collectives. Figure 8 shows the measured and modeled times for point-to-point (a and b) and collectives (c and d) for two configurations of miniAMR in sorted order of communication times. We observe similar performance predictions for P2P using all models. However, only DACM is able to model collectives well (less than 12% error). This is because DACM models the delays due to late arrival of ranks at collective calls, while the Basic, Max-rate and Congestion models only model the data transmission time. The measured time is less than 100 microseconds for about 10,000 collective calls (observe from start to 10,000 mark in the x-axis of the bottom two sub-plots). However, the communication time increases steeply for about 2,000 calls towards the end. This is due to late arrivals of ranks which is well-modeled by DACM. This shows the effectiveness of our modeling. On the contrary, Basic, Max-rate and Congestion are unable to model the delayed messages due to factors such as imbalance in arrival times of ranks at each collective invocation. Furthermore, since miniAMR is an adaptive application, its communication pattern may change across phases within the same run. However, the clustering algorithm can still provide the unique clusters that we use for performance modeling. Thus, the irregular nature of the application does not impact our modeling framework.

E. Fine-grained Analysis of Model Performance

We now show fine-grained comparisons of communication times using the measured, DACM and Basic model predictions for Quicksilver and miniAMR in Figure 9. We depict a short snapshot of traces (~ 100 seconds) for two different process counts. Distinct features of each application are visible as well as the well-defined repeated communication phases (timesteps)

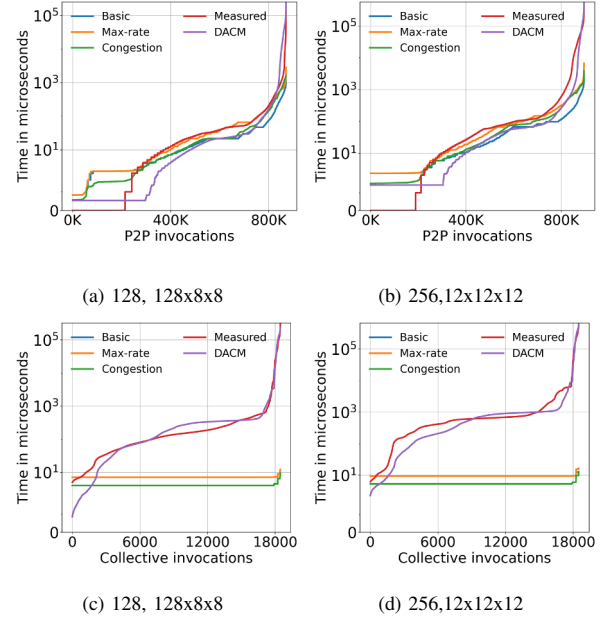


Fig. 8: Evaluation of model performance for individual collectives for two problem sizes of miniAMR, sorted in increasing order of time.

which were identified by our phase identification method. The gaps between two communication phases represent the computation phases. The detailed clusters of each timestep were shown in Section V-B. We observe that communication times of each MPI event range up to hundreds of milliseconds (red). We observe that DACM is able to predict well whereas Basic predicts less than 10 microseconds. We also observe that the function calls at the start of each phase (timestep) exhibit higher communication times (note the spike at the start of each communication phase). This may be due to the compute phase time variations leading to differences in arrival times of ranks for the collectives. We have observed this for all collectives in all configurations of all applications. However, DACM is able to handle these arrival delays and results in a maximum error of 34% and 24% across all configurations of Quicksilver and miniAMR, respectively.

F. End-to-end Robust Prediction

Now we present the compute time prediction results. We model the computation time using linear models similar to [7]. The sequential computation time is determined by the amount of *computation load* on a single rank. We scale the compute load per rank according to the problem size and process count. We use the measured computation time of a smaller problem size and process count as a reference and linearly scale it for different configurations [7]. Our experiments showed a minimum MAPE of 2.65% for LAMMPS, close to 10% for Quicksilver and HPCG, 20% for Xcompact and a maximum of 32% for miniAMR. The high error of miniAMR can be attributed to the irregular and dynamic nature of the application because it changes its behaviour across timesteps.

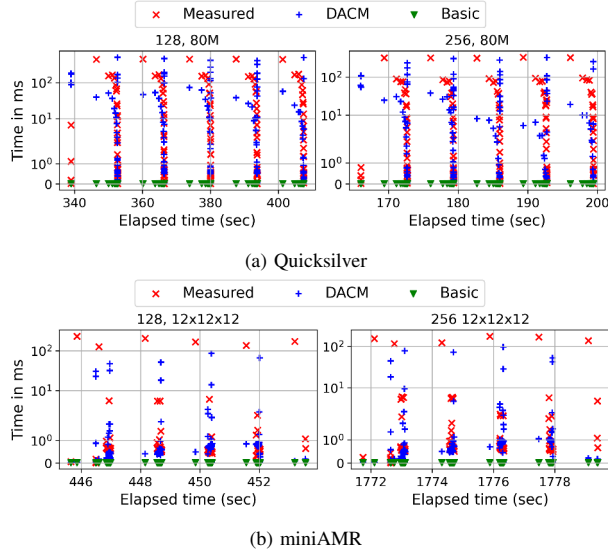


Fig. 9: Snapshot of traces showing measured and modeled collective communication time.

The compute time predictions added to communication time predictions will give an estimate of total execution times.

We also validated our models on a different system using the reference run from the same system as described in Section V-A. Each node of this new system has 16-core Intel Core i7 processors, 32 GB of memory, and connected with 1 Gbps Ethernet. We used the traces from our primary system. We ran the micro-benchmarks on this system to obtain network parameters (latency and bandwidth). For every application, we collected a single trace on the new system to predict performance of a new configuration using the DACM model. We tested the applications HPCG, LAMMPS, miniAMR and Quicksilver on 64, 128 and 256 processes. Our model achieved nearly 10% MAPE for both Quicksilver and miniAMR, 17% for HPCG and 49% for LAMMPS. The poor performance of LAMMPS is likely due to large P2P messages impacted by the different architecture and slower network. We plan to investigate this in future.

Further, we compared the accuracy of our DACM model with machine learning (ML) based regression models. Similar to [34], we used the input parameters and the process count as features and the total communication time as the label (output) of the ML model. Unlike traditional ML problems, we used 20% of the data for training and 80% for testing. We evaluated Extra Trees Regressor [23] and XGBoost [17] models; both models reported similar accuracy. LAMMPS, miniAMR, HPCG and Xcompact exhibited close to 50% MAPE while MAPE of Quicksilver was up to 77%. Other than Xcompact, DACM outperformed the ML models while using data from only a single instance of historical data. Note that analytical modeling using DACM does not require more than a few runs, in our case we used one reference run and either historical traces or traces of 1 timestep obtained from

a single node. This shows the efficacy of our phase detection and performance modeling approaches.

Finally, we observe that modeling only the unique clusters as opposed to the entire trace is up to $140\times$ faster across the different applications. While our Python implementation of the KMeans clustering algorithm takes up to 50% of our end-to-end modeling time, this is a one-time cost to obtain the clusters and thus can be avoided in repeated experimentations.

VI. CONCLUSIONS

In this paper, we proposed a novel way of modeling and analyzing HPC application performance using an abstraction of communication phases. We proposed a heuristic to automatically identify communication phases from traces of HPC applications. These phases were then mapped to unique and repeated communication clusters. This allowed us to model the performance of HPC applications by considering only the unique communication phases and using trace of only one timestep which may be obtained on a single node. Further, our new delay-aware communication model (DACM) addressed the limitation of deterministic analytical communication models by considering the difference in start times of MPI communications (especially collectives). DACM models the delays in communication time and achieves an average of 28% error for six applications on up to 320 processes. Our analytical models require a single reference run, whereas existing statistical approaches [7], [34] require at least 40% sampling data. They exhibit an average of more than 50% error with 20% sampling data. Excluding Xcompact, DACM achieved an average error of 17% for all configurations of NAS-LU, HPCG, LAMMPS, miniAMR, and Quicksilver. There are certain limitations of this work. We observed higher errors for certain applications such as Xcompact. This may have been due to non-blocking P2Ps. In this work, we modeled only blocking point-to-point and collective communications. We plan to model non-blocking communications in the future.

VII. ACKNOWLEDGMENTS

We acknowledge National Supercomputing Mission for providing computing resources of PARAM Sanganak at IIT Kanpur for this research work. This research work was partially supported by the Research-I Foundation of the Department of Computer Science and Engineering of IIT Kanpur.

REFERENCES

- [1] Coral-2 benchmarks. <https://asc.llnl.gov/coral-2-benchmarks>.
- [2] Extrac. <https://tools.bsc.es/extrac>.
- [3] Intel® vtune™ profiler. <https://www.intel.com/content/www/us/en/developer/tools/oneapi/vtune-profiler.html#gs.n8x52c>.
- [4] Xcompact3d. <https://xcompact3d.readthedocs.io/en/latest/>.
- [5] Omar Aaziz, Mohammad Al-Tahat, Strahinja Trecakov, and Jonathan Cook. Incprof: Efficient source-oriented phase identification for application behavior understanding. In *2022 IEEE International Conference on Cluster Computing (CLUSTER)*, pages 616–625, 2022.
- [6] Laksono Adhianto, S Banerjee, M. Fagan, M Krentel, Gabriel Marin, John Mellor-Crummey, and Nathan Tallent. Hpctoolkit: Tools for performance analysis of optimized parallel programs. *Concurrency and Computation: Practice and Experience*, 22, 12 2009.

- [7] Shivam Aggarwal and Preeti Malakar. Evaluating active-learning based performance prediction of parallel applications. In *2024 IEEE 20th International Conference on e-Science (e-Science)*, pages 1–10, 2024.
- [8] Tushar Agrawal and Preeti Malakar. Ipmi: Improved mpi communication logger. In *2022 IEEE/ACM International Workshop on Exascale MPI (ExaMPI)*, pages 31–40, 2022.
- [9] Xavier Aguilar, Karl F rlinger, and Erwin Laure. Mpi trace compression using event flow graphs. In *Euro-Par 2014 Parallel Processing: 20th International Conference, Porto, Portugal, August 25-29, 2014. Proceedings 20*, pages 1–12. Springer, 2014.
- [10] Luay Alawneh and Abdelwahab Hamou-Lhadj. Identifying computational phases from inter-process communication traces of hpc applications. In *2012 20th IEEE International Conference on Program Comprehension (ICPC)*, pages 133–142, 2012.
- [11] Albert Alexandrov, Mihai F. Ionescu, Klaus E. Schauser, and Chris Scheiman. Loggp: incorporating long messages into the logp model—one step closer towards a realistic model for parallel computation. In *Proceedings of the Seventh Annual ACM Symposium on Parallel Algorithms and Architectures, SPAA ’95*, page 95–105, New York, NY, USA, 1995. Association for Computing Machinery.
- [12] David H Bailey, Eric Barszcz, John T Barton, David S Browning, Robert L Carter, Leonardo Dagum, Rod A Fatoohi, Paul O Frederickson, Thomas A Lasinski, Rob S Schreiber, et al. The nas parallel benchmarks—summary and preliminary results. In *Proceedings of the 1991 ACM/IEEE Conference on Supercomputing*, pages 158–165, 1991.
- [13] Kevin J. Barker, Kei Davis, Adolfo Hoisie, Darren J. Kerbyson, Michael Lang, Scott Pakin, and Jos  Carlos Sancho. Using performance modeling to design large-scale systems. *Computer*, 42(11):42–49, 2009.
- [14] Bradley J. Barnes, Barry Rountree, David K. Lowenthal, Jaxk Reeves, Bronis de Supinski, and Martin Schulz. A regression-based approach to scalability prediction. In *Proceedings of the 22nd Annual International Conference on Supercomputing, ICS ’08*, page 368–377, New York, NY, USA, 2008. Association for Computing Machinery.
- [15] Greg Bauer, Steven Gottlieb, and Torsten Hoefer. Performance modeling and comparative analysis of the milc lattice qcd application su3_rmd. In *2012 12th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (ccgrid 2012)*, pages 652–659, 2012.
- [16] Arnaboy Bhattacharyya and Torsten Hoefer. Pemogen: Automatic adaptive performance modeling during program runtime. In *2014 23rd International Conference on Parallel Architecture and Compilation Techniques (PACT)*, pages 393–404, 2014.
- [17] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pages 785–794, 2016.
- [18] Jaemin Choi, David F. Richards, Laxmikant V. Kale, and Abhinav Bhatele. End-to-end performance modeling of distributed gpu applications. In *Proceedings of the 34th ACM International Conference on Supercomputing, ICS ’20*, New York, NY, USA, 2020. Association for Computing Machinery.
- [19] Jason Cope, Ning Liu, Samuel Lang, Philip H. Carns, Christopher D. Carothers, and Robert B. Ross. Codes: Enabling co-design of multi-layer exascale storage architectures. 2011.
- [20] Paul Stewart Crozier, Heidi K Thornquist, et al. Improving performance via mini-applications. Technical report, Sandia National Laboratories (SNL), Albuquerque, NM, and Livermore, CA . . . , 2009.
- [21] David Culler, Richard Karp, David Patterson, Abhijit Sahay, Klaus Erik Schauser, Eunice Santos, Ramesh Subramonian, and Thorsten von Eicken. Logp: towards a realistic model of parallel computation. *SIGPLAN Not.*, 28(7):1–12, jul 1993.
- [22] Hormozd Gahvari, Allison H. Baker, Martin Schulz, Ulrike Meier Yang, Kirk E. Jordan, and William Gropp. Modeling the performance of an algebraic multigrid cycle on hpc platforms. ICS ’11, page 172–181, New York, NY, USA, 2011. Association for Computing Machinery.
- [23] Pierre Geurts, Damien Ernst, and Louis Wehenkel. Extremely randomized trees. *Machine learning*, 63:3–42, 2006.
- [24] Sergi Girona and Jes  Labarta. Sensitivity of performance prediction of message passing programs. 17(3):291–298, nov 2000.
- [25] William Gropp, Luke N. Olson, and Philipp Samfass. Modeling mpi communication performance on smp nodes: Is it time to retire the ping pong test. In *Proceedings of the 23rd European MPI Users’ Group Meeting, EuroMPI ’16*, page 41–50, New York, NY, USA, 2016. Association for Computing Machinery.
- [26] Michael Allen Heroux, Jack Dongarra, and Piotr Luszczek. Hpcg benchmark technical specification. Technical report, Sandia National Lab.(SNL-NM), Albuquerque, NM (United States), 2013.
- [27] Roger W. Hockney. The communication challenge for mpp: Intel paragon and meiko cs-2. *Parallel Computing*, 20(3):389–398, 1994.
- [28] Torsten Hoefer, Timo Schneider, and Andrew Lumsdaine. LogGOPSim - Simulating Large-Scale Applications in the LogGOPS Model. In *Proceedings of the 19th ACM International Symposium on High Performance Distributed Computing*, pages 597–604. ACM, Jun. 2010.
- [29] Sascha Hunold, Jordy I. Ajanohoun, Ioannis Vardas, and Jesper Larsson Tr ff. An overhead analysis of mpi profiling and tracing tools. In *Proceedings of the 2nd Workshop on Performance Engineering, Modelling, Analysis, and Visualization Strategy, PERMAVOST ’22*, page 5–13, 2022.
- [30] Engin Ipek, Bronis R. de Supinski, Martin Schulz, and Sally A. McKee. An approach to performance prediction for parallel applications. In Jos  C. Cunha and Pedro D. Medeiros, editors, *Euro-Par 2005 Parallel Processing*, pages 196–205, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.
- [31] Katherine E. Isaacs, Todd Gamblin, Abhinav Bhatele, Martin Schulz, Bernd Hamann, and Peer-Timo Bremer. Ordering traces logically to identify lateness in message passing programs. *IEEE Transactions on Parallel and Distributed Systems*, 27(3):829–840, 2016.
- [32] Ye Jin, Xiaosong Ma, Mingliang Liu, Qing Liu, Jeremy Logan, Norbert Podhorski, Jong Youl Choi, and Scott Klasky. Combining phase identification and statistic modeling for automated parallel benchmark generation. In *Proceedings of the 2015 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, SIGMETRICS ’15*, page 309–320, New York, NY, USA, 2015. Association for Computing Machinery.
- [33] Andreas Kn pfer, Holger Brunst, Jens Doleschal, Matthias Jurenz, Matthias Lieber, Holger Mickler, Matthias M ller, and Wolfgang Nagel. The vampir performance analysis tool-set. pages 139–155, 01 2008.
- [34] Preeti Malakar, Prasanna Balaprakash, Venkatram Vishwanath, Vitali Morozov, and Kalyan Kumar. Benchmarking machine learning methods for performance modeling of scientific applications. In *2018 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, pages 33–44, 2018.
- [35] D. R. Martinez, J. C. Cabaleiro, T. F. Pena, F. F. Rivera, and V. Blanco. Accurate analytical performance model of communications in MPI applications. In *2009 IEEE International Symposium on Parallel & Distributed Processing*, 2009.
- [36] Jelena Pje ivac-Grbovi , Thara Angskun, George Bosilca, Graham E. Fagg, Edgar Gabriel, and Jack J. Dongarra. Performance analysis of mpi collective operations. *Cluster Computing*, 10(2):127–143, Jun 2007.
- [37] A. F. Rodrigues, K. S. Hemmert, B. W. Barrett, C. Kersey, R. Oldfield, M. Weston, R. Risen, J. Cook, P. Rosenfeld, E. Cooper-Balis, and B. Jacob. The structural simulation toolkit. 38(4):37–42, mar 2011.
- [38] Arun Rodrigues, K. Hemmert, Brian Barrett, Chad Kersey, Ron Oldfield, M. Weston, R. Risen, Jeanine Cook, Paul Rosenfeld, E. CooperBalls, and Bruce Jacob. The structural simulation toolkit. *ACM SIGMETRICS Performance Evaluation Review*, 38:37–42, 03 2011.
- [39] Siyuan Shen, Langwen Huang, Marcin Chrapek, Timo Schneider, Jai Dayal, Manisha Gajbe, Robert Wisniewski, and Torsten Hoefer. Llmp: Assessing network latency tolerance of hpc applications with linear programming. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis, SC ’24*. IEEE Press, 2024.
- [40] Sameer S. Shende and Allen D. Malony. The tau parallel performance system. *Int. J. High Perform. Comput. Appl.*, 20(2):287–311, 2006.
- [41] A. Snively, X. Gao, C. Lee, L. Carrington, N. Wolter, J. Labarta, J. Gimenez, and P. Jones. Performance modeling of HPC applications. In G.R. Joubert, W.E. Nagel, F.J. Peters, and W.V. Walter, editors, *Parallel Computing*, volume 13 of *Advances in Parallel Computing*, pages 777–784. North-Holland, 2004.
- [42] David Sundaram-Stukel and Mary K. Vernon. Predictive analysis of a wavefront application using loggp. In *Proceedings of the Seventh ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPOPP ’99*, page 141–150, New York, NY, USA, 1999. Association for Computing Machinery.
- [43] Rajeev Thakur and William D. Gropp. Improving the performance of collective operations in mpich. In Jack Dongarra, Domenico Laforenza, and Salvatore Orlando, editors, *Recent Advances in Parallel Virtual*

- Machine and Message Passing Interface*, pages 257–267, Berlin, Heidelberg, 2003. Springer Berlin Heidelberg.
- [44] Rajeev Thakur, Rolf Rabenseifner, and William Gropp. Optimization of collective communication operations in mpich. *The International Journal of High Performance Computing Applications*, 19(1):49–66, 2005.
 - [45] Aidan P Thompson, H Metin Aktulga, Richard Berger, Dan S Bolinteanu, W Michael Brown, Paul S Crozier, Pieter J In't Veld, Axel Kohlmeyer, Stan G Moore, Trung Dac Nguyen, et al. Lammmps-a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales. *Computer Physics Communications*, 271:108171, 2022.
 - [46] Zhou Tong, Scott Pakin, Michael Lang, and Xin Yuan. Fast classification of mpi applications using lamport's logical clocks. In *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 618–627, 2016.
 - [47] Jeffrey Vetter and Chris Chembreau. mpiP: Lightweight, scalable MPI profiling.
 - [48] Felix Wolf, Brian Wylie, Erika Ábrahám, Daniel Becker, Wolfgang Frings, Karl Furlinger, Markus Geimer, Marc-André Hermanns, Bernd Mohr, Shirley Moore, Matthias Pfeifer, and Zoltán Szebenyi. Usage of the scalasca toolset for scalable performance analysis of large-scale parallel applications. pages 157–167, 01 2008.
 - [49] Tao Yan, Qingguo Xu, Jiyu Luo, Jingwei Sun, and Guangzhong Sun. Scalable tracing of mpi events and performance metrics. In *2023 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 714–723, 2023.